

개발기관	2025-XXXX-XX
배정연번	

개발이력	
최초 개발	2025-11-01
1차 업데이트	1.
2차 업데이트	2.
3차 업데이트	3.

「 AI 제조 기술 교육-25-XX 」

OrderBean_PRD_v1.md

제 출 일 :

소 속 :

성 명 :

※ 워크북 제출시 반드시 파일명을 ○○○○(제출년월일8자리)_○○○(성명).hwp로 하여
주십시오.(예시 : 20250128_홍길동.hwp)

개발일자 : 2025-11-01



OrderBean_PRD_v1.md

버전: 1.0

프로젝트명: OrderBean Toy Project

작성일: 2025-10-21

작성자: 풀스택 웹 AI 부트캠프 팀

1. 개요 (Overview)

OrderBean은 웹 기반 커피 주문 시스템으로, 사용자가 웹 페이지를 통해 메뉴를 확인하고, 옵션을 선택하여 주문 및 결제를 완료할 수 있도록 하는 풀스택 웹 프로젝트입니다.

본 시스템은 **FastAPI(백엔드)**, **SQLite(데이터베이스)**, ****HTML/JavaScript(프론트엔드)****를 기반으로 개발되며,

AI 도구(**Cursor.AI**)를 활용하여 설계부터 테스트까지 자동화된 개발 프로세스를 시연합니다.

프로젝트 목표

- 간편하고 직관적인 커피 주문 경험 제공
- FastAPI를 활용한 풀스택 웹 서비스 구현
- AI 보조 코딩(Cursor.AI) 기반 자동화 개발 프로세스 실습
- 클린코드, 테스트, 리팩토링 개념을 실제 프로젝트에 적용

2. 주요 사용자 (Target Users)

사용자 유형	설명	주요 목표
고객(User)	커피를 주문하는 일반 사용자	메뉴 확인, 옵션 선택, 주문 및 결제
관리자(Admin)	매장 관리자 또는 바리스타	메뉴 등록·수정·삭제, 주문 관리

3. 사용자 스토리 (User Stories - Gherkin 형식)

기능: 커피 주문

커피를 좋아하는 사용자로서
내가 원하는 커피를 선택하고 옵션을 지정하여 주문하고 싶다.
그래야 편리하게 주문을 완료할 수 있다.

시나리오: 정상적인 커피 주문

주어진 조건: 사용자가 로그인 상태일 때
언제: 사용자가 커피를 선택하고 옵션을 지정하고 결제 버튼을 클릭하면
그러면: 주문이 저장되고 상태가 "준비중"으로 표시된다.

기능: 쿠폰 할인

단골 고객으로서
쿠폰 코드를 입력하여 할인 혜택을 받고 싶다.

시나리오: 유효한 쿠폰 사용

주어진 조건: 사용자가 유효한 쿠폰을 가지고 있을 때
언제: 쿠폰 코드를 입력하면
그러면: 주문 금액에 할인율이 적용되어 최종 금액이 계산된다.

4. 기능 요구사항 (Functional Requirements)

ID	요구사항	설명	우선순 위	수용 기준
FR-01	사용자 인증	회원가입, 로그인, 로그아웃 기능 제공	높음	JWT 인증을 이용한 로그인 구현
FR-02	메뉴 조회	메뉴 목록(이미지, 이름, 가격) 표시	높음	/api/menu에서 메뉴 데이터 반환

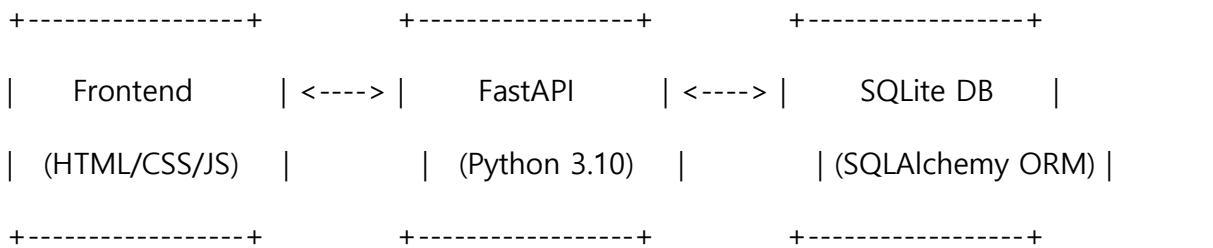
ID	요구사항	설명	우선순 위	수용 기준
FR-03	주문 생성	사용자가 메뉴 선택 및 옵션 지정 후 주문 가능	높음	주문 정보가 DB에 저장됨
FR-04	쿠폰 적용	쿠폰 코드를 통해 할인 적용	중간	유효한 쿠폰 검증 및 금액 반영
FR-05	주문 내역 조회	사용자 주문 이력 확인 기능	중간	/api/order/history에서 목록 반환
FR-06	관리자 메뉴 관리	관리자가 메뉴 등록/수정/삭제 가능	낮음	관리자 권한만 접근 가능

5. 비기능 요구사항 (Non-Functional Requirements)

항목	설명
성능(Performance)	50명 동시 접속 시 응답 시간 1초 이하
보안(Security)	비밀번호 암호화(bcrypt), JWT 인증 적용
확장성(Scalability)	모듈화 구조 설계로 기능 확장 용이
사용성(Usability)	모바일 대응형 반응형 웹, 심플한 인터페이스
유지보수성(Maintainability)	PEP8, 클린코드 원칙, 테스트 커버리지 85% 이상

6. 시스템 구조 (System Architecture)

6.1 아키텍처 개요



- **Frontend:** HTML + JS를 이용한 기본 웹 화면
- **Backend:** FastAPI RESTful API
- **Database:** SQLite (SQLAlchemy ORM 이용)
- **Testing:** Pytest + httpx 활용

7. 데이터 모델 (ERD 설명)

테이블	필드	설명
User	id(PK), name, email, password_hash, role	사용자 정보 저장
Menu	id(PK), name, category, price, image_url	메뉴 항목 정보
Order	id(PK), user_id(FK), total_price, created_at, status	주문 내역
OrderItem	id(PK), order_id(FK), menu_id(FK), quantity, size, options	주문 상세 항목
Coupon	id(PK), code, discount_percent, expiry_date, is_active	쿠폰 코드 관리

8. API 명세 (API Specification)

엔드포인트	메서드	설명	인증	요청 예시	응답 예시
/api/auth/signup	POST	회원가입	X	{name, email, password}	{user_id, token}
/api/auth/login	POST	로그인	X	{email, password}	{token}
/api/menu	GET	메뉴 조회	X	-	[{id, name, price, image_url}]
/api/order	POST	주문 생성	O	{items:[], coupon_code}	{order_id, total_price}
/api/order/history	GET	주문 내역 조회	O	-	[{order_id, total_price, date, status}]

엔드포인트	메서드	설명	인종	요청 예시	응답 예시
/api/coupon/validate	POST	쿠폰 유효성 확인	O	{code}	{valid:true, discount_percent}

9. UX 흐름 (UX Flow)

[홈] → [로그인/회원가입] → [메뉴 보기] → [메뉴 선택] → [옵션 지정]

↓

[쿠폰 입력] → [주문 요약] → [결제 완료] → [주문 확인 페이지]

- 컬러 테마: AI Blue (#1E90FF), 화이트, 라이트그레이
- 폰트: Inter / Noto Sans KR
- 디자인 스타일: 단순하고 반응형 UI

10. 테스트 계획 (Test Plan)

테스트 ID	내용	유형	기대 결과
TC-01	회원가입 및 로그인 테스트 단위(Unit)		JWT 토큰 반환
TC-02	유효하지 않은 쿠폰 입력	단위(Unit)	400 Bad Request
TC-03	주문 생성 테스트	통합(Integration)	DB에 주문 정보 저장
TC-04	관리자 메뉴 등록 기능	통합(Integration)	신규 메뉴 GET /api/menu에 표시
TC-05	메뉴 조회 속도 테스트	성능	1초 이하 응답
TC-06	테스트 커버리지 측정	품질	85% 이상 통과

11. 수용 기준 (Acceptance Criteria)

- 모든 API가 정상 동작 및 유효한 JSON 반환

- 비즈니스 로직(쿠폰, 주문, 결제)이 요구사항대로 작동
- 모든 테스트(Pytest)가 통과하며 커버리지 85% 이상
- /healthcheck 응답 200 OK
- README.md에 설치 및 실행 방법 명시

12. 향후 개선 계획 (Future Enhancements)

ID	기능	설명	우선순위
FE-01	챗봇 주문	AI 챗봇을 통한 대화형 주문 기능 추가	중간
FE-02	결제 시스템 연동	Stripe 또는 PayPal 연동	높음
FE-03	포인트 적립	주문 누적 포인트 제도 도입	중간
FE-04	추천 시스템	과거 주문 기반 메뉴 추천 기능	낮음

13. 참고 문서 (References)

- FastAPI 공식 문서
- SQLAlchemy ORM 문서
- Pytest 테스트 가이드
- 클린코드 - Robert C. Martin 저

✅ 요약 (Summary)

항목	내용
문서 유형	제품 요구사항 문서(PRD)
버전	1.0
프로젝트 목적	풀스택 웹 개발 실습용 토이 프로젝트

항목

내용

핵심 기술

FastAPI, SQLite, Pytest, Cursor.AI

최종 목표

AI 보조 개발을 통한 주문 웹 서비스 구현 및 배포

Cursor.AI에서 실제 프로젝트를 단계별로 완성할 수 있는 Prompt

이 세트는 "AI 개발 파이프라인"에 따라

📄 PRD → 🏗️ 설계 → 💻 구현 → 🧪 테스트 → 🧠 리팩토링 → 🚀 배포

순으로 설계되어 있으며,

각 프롬프트는 Cursor.AI의 대화형 코드 생성 콘솔에 바로 붙여넣어 사용할 수 있습니다.

🍵 OrderBean Cursor Prompt Set

기반 문서: OrderBean_PRD_v1.md

목적: Cursor.AI를 활용하여 OrderBean 풀스택 웹 서비스를 자동 생성 및 개선하기

🚀 전체 흐름

단계 목적	주요 결과물
① PRD 분석 및 설계	ERD, 아키텍처 구조, API 정의
② 백엔드 코드 생성	FastAPI + SQLAlchemy + CRUD
③ 테스트 자동화	Pytest 기반 단위/통합 테스트
④ 리팩토링	SOLID, 중복 제거, 품질 개선
⑤ 배포 자동화	GitHub Actions + Render 배포
⑥ 문서화 및 회고	README + 기술 문서 + 보고서

🌱 STEP 1 — PRD 분석 및 설계 자동화

목표: AI가 OrderBean의 핵심 구조를 이해하고 설계 문서를 자동으로 생성하도록 한다.

◆ Prompt 1: PRD 요약 및 핵심 기능 추출

You are an AI software architect.

Read the following PRD (OrderBean_PRD_v1.md) and summarize:

1. The main entities (User, Menu, Order, etc.)
2. The user roles and key use cases
3. The main functional and non-functional requirements
4. The overall data flow (client → API → DB)

Output as a short architecture summary.

◆ Prompt 2: ERD 자동 생성

Based on the PRD, generate an Entity Relationship Diagram (ERD)

for the OrderBean system including User, Menu, Order, OrderItem, and Coupon tables.

Describe each relationship (1:N, N:N) in text and output SQLAlchemy model schema.

◆ Prompt 3: 시스템 아키텍처 다이어그램

Generate a fullstack architecture diagram for the OrderBean project.

Include frontend (HTML/JS), backend (FastAPI), and database (SQLite).

Explain each layer's role and main communication flow.

◆ Prompt 4: API 설계 명세

From the PRD, generate a RESTful API specification table including:

- Endpoint
- Method
- Description

- Request schema
- Response schema

Follow FastAPI conventions.

STEP 2 — FastAPI 코드 자동 생성

목표: 설계 정보를 기반으로 **FastAPI 백엔드 코드**를 자동으로 작성한다.

◆ Prompt 5: 프로젝트 기본 구조 생성

Create a FastAPI project folder structure for OrderBean:

app/

```
|— main.py
|— routers/
|— models/
|— services/
|— schemas/
|— tests/
```

Include `__init__.py` where necessary.

◆ Prompt 6: 데이터 모델 생성

Generate SQLAlchemy models for User, Menu, Order, OrderItem, and Coupon based on the PRD ERD. Include relationships, `__repr__` methods, and default timestamps.

◆ Prompt 7: API 라우터 생성

Create FastAPI routers for the following endpoints:

- `/api/auth` (signup, login)

- /api/menu (GET, POST)
- /api/order (POST, GET)
- /api/coupon (POST validate)

Use Pydantic schemas for input/output validation.

◆ Prompt 8: 비즈니스 로직 구현

Implement service functions for order creation:

- Calculate subtotal, discount, total price
- Apply coupon validation if exists
- Save order and order items to DB

Include error handling and logging.

STEP 3 — 테스트 코드 및 품질 검증

목표: Pytest를 이용해 자동 테스트와 커버리지 확보.

◆ Prompt 9: 테스트 코드 자동 생성

Generate Pytest test cases for the OrderBean project:

- Unit tests for menu, order, and coupon logic
- Integration tests for API endpoints (/api/order)
- Use Faker for fake data generation

Follow Given/When/Then structure in docstrings.

◆ Prompt 10: 커버리지 분석 및 개선

Run pytest with coverage and identify untested lines.

Generate new test cases to cover missing branches and edge cases.

Target coverage: $\geq 85\%$.

◆ Prompt 11: Clean Code 리뷰

Review the generated code for Clean Code principles:

- Meaningful variable names
- Short functions
- No duplication
- Consistent error handling

Suggest specific improvements with examples.

🧠 STEP 4 — 리팩토링 및 코드 개선

목표: SOLID 원칙과 클린 아키텍처 적용.

◆ Prompt 12: 코드 스멜 탐지

Identify code smells in the current codebase

(e.g., long methods, duplicate logic, feature envy, large class).

Suggest refactoring strategies for each case.

◆ Prompt 13: SOLID 리팩토링

Refactor the OrderBean service layer according to SOLID principles:

- Split large functions into smaller ones
- Apply dependency injection
- Keep all tests passing

Explain the reasoning behind each change.

◆ Prompt 14: 구조 개선 (폴더 리팩토링)

Restructure the OrderBean project to follow Clean Architecture:

app/

├── api/

├── domain/

├── repositories/

├── services/

└── tests/

Update import paths accordingly.

◆ Prompt 15: 코드 리뷰 자동화

Generate a code review summary highlighting:

- Readability issues
- Performance concerns
- Security risks

Format comments as if written by a senior reviewer.

⚙️ STEP 5 — CI/CD 및 배포 자동화

목표: Render 및 GitHub Actions를 활용하여 배포 자동화 구축.

◆ Prompt 16: CI/CD 파이프라인 생성

Create a GitHub Actions workflow file (.github/workflows/ci.yml)

that performs the following on push to main:

- Install dependencies
- Run pytest

- Run black (code formatting check)
 - Deploy to Render if tests pass
-

◆ Prompt 17: 환경 변수 설정

Generate .env example file including:

DATABASE_URL, JWT_SECRET, RENDER_API_KEY

and update main.py to load environment variables using python-dotenv.

◆ Prompt 18: Render 배포 스크립트

Generate Render deployment configuration:

- Python 3.10
- Start command: uvicorn app.main:app --host 0.0.0.0 --port 8000

Include build and deploy steps in render.yaml.

STEP 6 — 문서화 및 회고

목표: 프로젝트 문서 및 학습 결과 정리.

◆ Prompt 19: README 자동 생성

Generate a README.md for the OrderBean project including:

- Overview
- Setup instructions
- API endpoints
- Test commands
- Deployment instructions
- Contributors

◆ Prompt 20: 프로젝트 회고 보고서

Summarize what was learned from developing the OrderBean system with AI.

Include key takeaways for:

1. Prompt Engineering
2. Clean Code
3. Refactoring
4. Test Automation
5. CI/CD Deployment

Output as Markdown for documentation.

✅ 전체 결과물 (Expected Outputs)

단계 산출물

Step 1 ERD, API 명세, 시스템 아키텍처

Step 2 FastAPI 백엔드 코드 + 데이터 모델

Step 3 Pytest 테스트 세트 + 커버리지 리포트

Step 4 리팩토링된 코드 + 리뷰 로그

Step 5 GitHub Actions + Render 배포 파일

Step 6 README.md + 회고 문서

💡 사용 팁

- **Prompt 체인 방식**으로 실행하세요.
→ Prompt 1 → Prompt 2 → Prompt 3 ... 순차적으로 붙여넣으면
Cursor가 이전 맥락을 기억해 일관된 프로젝트를 생성합니다.
- **중간 확인 프롬프트:**

- Summarize current project progress and show directory tree.

로 진행 상황을 점검하세요.

- **Refactor 요청 시 주의:**

“tests must remain green” 조건을 반드시 포함해야 합니다.

(AI가 테스트를 깨뜨리지 않도록 유도)